

Ex. No: 07

Lane Detection using Hough Transform.

Name : ASWINI D **Reg. No :** 212225240015 **Slot No :** T1-H15

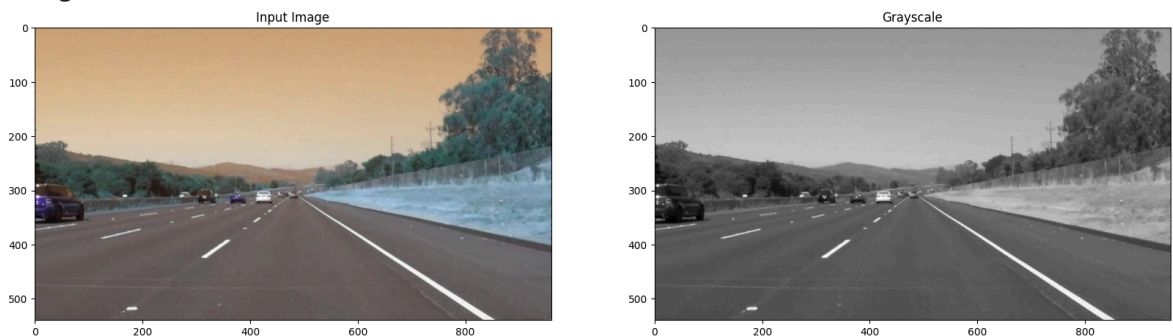
```
In [1]: # Import
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: # Read the image using OpenCV
####
image = cv2.imread('lan_img1.jpg')
####
```

```
In [3]: # Convert to grayscale.
####
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
####
```

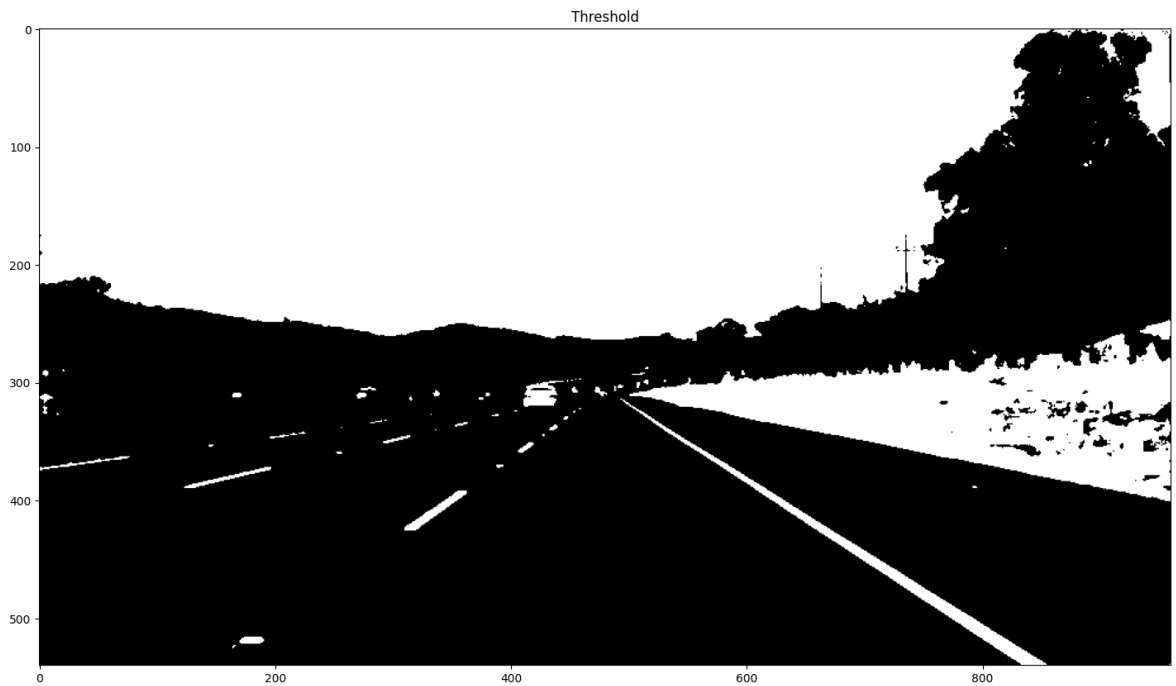
```
In [4]: #Output
plt.figure(figsize = (15, 10))
plt.figure(figsize = (20, 10))
plt.subplot(1,2,1); plt.imshow(image);plt.title('Input Image');
plt.subplot(1,2,2); plt.imshow(gray_image, cmap = 'gray');plt.title('Grayscale')
```

<Figure size 1500x1000 with 0 Axes>



```
In [5]: # Use global threshold based on grayscale intensity.
threshold = cv2.inRange(gray_image, 150, 255)
```

```
In [6]: # Expected Output
# Display images.
plt.figure(figsize = (20, 10))
plt.subplot(1,1,1); plt.imshow(threshold, cmap = 'gray'); plt.title('Threshold')
```



```
In [7]: # Region masking: Select vertices according to the input image.
roi_vertices = np.array([[[100, 540],
                           [900, 540],
                           [515, 320],
                           [450, 320]]])

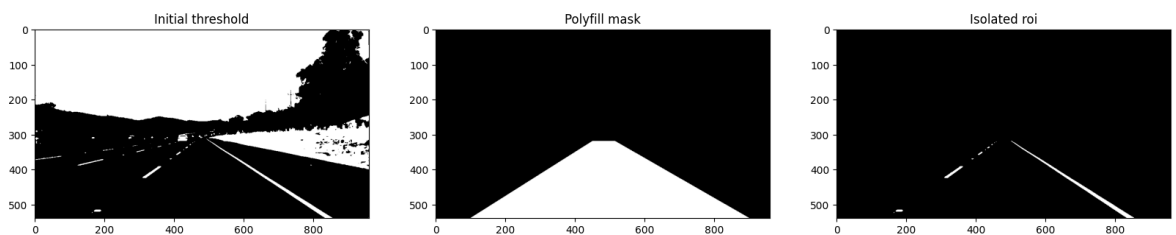
# Defining a blank mask.
mask = np.zeros_like(threshold)

# Defining a 3 channel or 1 channel color to fill the mask.
if len(threshold.shape) > 2:
    channel_count = threshold.shape[2] # 3 or 4 depending on the image.
    ignore_mask_color = (255,) * channel_count
else:
    ignore_mask_color = 255

# Filling pixels inside the polygon.
cv2.fillPoly(mask, roi_vertices, ignore_mask_color)

# Constructing the region of interest based on where mask pixels are nonzero.
roi = cv2.bitwise_and(threshold, mask)

# Display images.
plt.figure(figsize = (20, 10))
plt.subplot(1,3,1); plt.imshow(threshold, cmap = 'gray'); plt.title('Initial thr
plt.subplot(1,3,2); plt.imshow(mask, cmap = 'gray');      plt.title('Polyfill ma
plt.subplot(1,3,3); plt.imshow(roi, cmap = 'gray');       plt.title('Isolated ro
```



```
In [8]: # Perform Edge Detection (using Canny)
####
low_threshold = 50
```

```

high_threshold = 100
edges = cv2.Canny(roi, low_threshold, high_threshold)
###

```

```

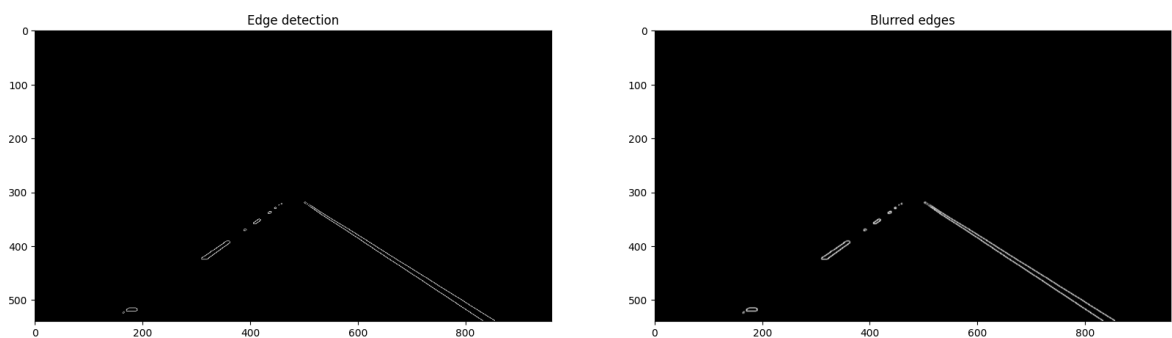
In [9]: # Smooth with a Gaussian blur.
###
kernel_size = 3
canny_blur = cv2.GaussianBlur(edges, (kernel_size, kernel_size), 0)
###

```

```

In [10]: #Expected Output
# Display images.
plt.figure(figsize = (20, 10))
plt.subplot(1,2,1); plt.imshow(edges, cmap = 'gray'); plt.title('Edge detection')
plt.subplot(1,2,2); plt.imshow(canny_blur, cmap = 'gray'); plt.title('Blurred ed

```



```

In [11]: def draw_lines(img, lines, color=(0, 255, 0), thickness=2):
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line
            cv2.line(img, (x1, y1), (x2, y2), color, thickness)

```

```

In [12]: # Hough transform parameters set according to the input image.
###
rho = 1
theta = np.pi / 180
threshold = 50
min_line_len = 10
max_line_gap = 20
lines = cv2.HoughLinesP(
    canny_blur, rho, theta, threshold, minLineLength = min_line_len, maxLineGap
###

```

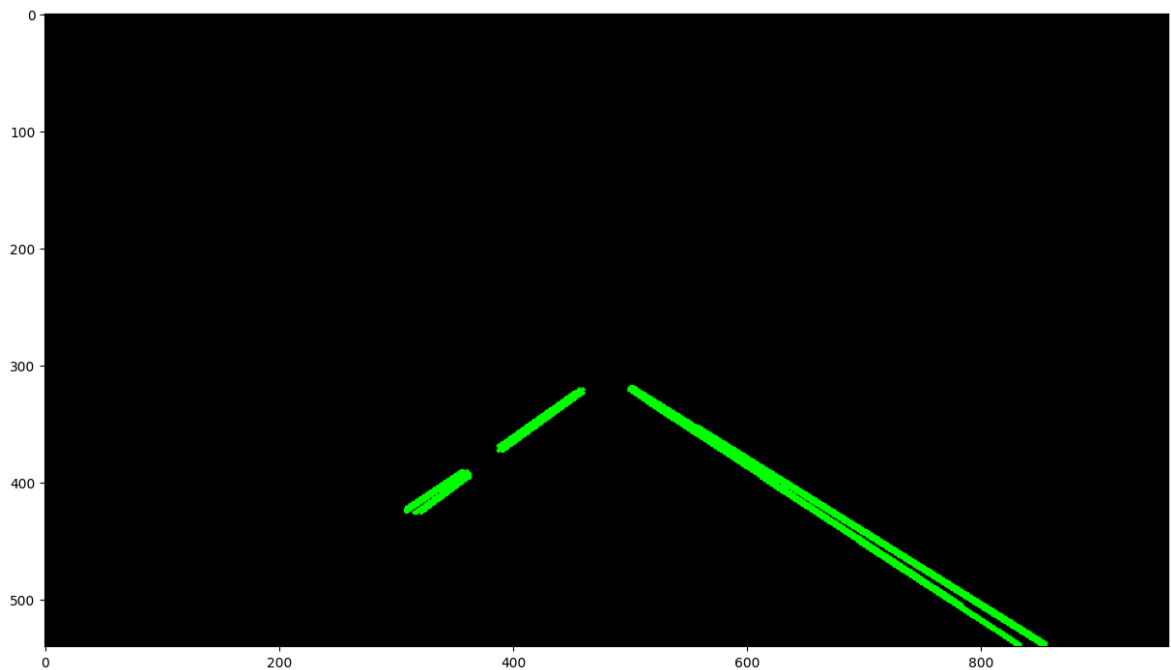
```

In [13]: # Expected output
# Draw all lines found onto a new image.
hough = np.zeros((image.shape[0], image.shape[1], 3), dtype = np.uint8)
draw_lines(hough, lines)

print("Found {} lines, including: {}".format(len(lines), lines[0]))
plt.figure(figsize = (15, 10)); plt.imshow(hough);

```

Found 21 lines, including: [500 318 853 539]



```
In [14]: def separate_left_right_lines(lines):
left_lines = []
right_lines = []

if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = np.array(line).reshape(-1)

        if y1 > y2: # Negative slope = Left lane
            left_lines.append([x1, y1, x2, y2])
        else:       # Positive slope = right lane
            right_lines.append([x1, y1, x2, y2])

    return left_lines, right_lines
```

```
In [15]: def cal_avg(values):
        """Calculate average value."""
        if not (type(values) == 'NoneType'):
            if len(values) > 0:
                n = len(values)
            else:
                n = 1
            return sum(values) / n
```

```
In [16]: def extrapolate_lines(lines, upper_border, lower_border):
        """Extrapolate lines keeping in mind the lower and upper border intersection
        slopes = []
        consts = []

        if lines is not None:
            for x1, y1, x2, y2 in lines:
                slope = (y1-y2) / (x1-x2)
                slopes.append(slope)
                c = y1 - slope * x1
                consts.append(c)
        avg_slope = cal_avg(slopes)
        avg_consts = cal_avg(consts)
```

```

# Calculate average intersection at lower_border.
x_lane_lower_point = int((lower_border - avg_consts) / avg_slope)

# Calculate average intersection at upper_border.
x_lane_upper_point = int((upper_border - avg_consts) / avg_slope)

return [x_lane_lower_point, lower_border, x_lane_upper_point, upper_border]

```

```

In [17]: # Define bounds of the region of interest.
roi_upper_border = 340
roi_lower_border = 540

# Create a blank array to contain the (colored) results.
lanes_img = np.zeros((image.shape[0], image.shape[1], 3), dtype = np.uint8)

# Use above defined function to identify lists of left-sided and right-sided lines
lines_left, lines_right = separate_left_right_lines(lines)

# Use above defined function to extrapolate the lists of lines into recognized lanes
lane_left = extrapolate_lines(lines_left, roi_upper_border, roi_lower_border)
lane_right = extrapolate_lines(lines_right, roi_upper_border, roi_lower_border)
draw_lines(lanes_img, [lane_left], thickness=10)
draw_lines(lanes_img, [lane_right], thickness=10)

# Display results.
fig = plt.figure(figsize = (20, 20))
ax = fig.add_subplot(1, 2, 1); plt.imshow(hough); ax.set_title('Before extrapolation')
ax = fig.add_subplot(1, 2, 2); plt.imshow(lanes_img); ax.set_title('After extrapolation')
plt.show()

```

